Clase 13: Sockets TCP

Introducción: la abstracción que ganó

En la clase anterior usamos `nc` para hablar con servidores y levantar uno propio. Ahora vamos a escribir eso nosotros.

La herramienta es el **socket**, y conviene entender de dónde salió, porque explica por qué la API tiene la forma rara que tiene.

Un poco de historia

A fines de los 70, ARPANET funcionaba pero programarla era un suplicio: cada fabricante ofrecía su propia interfaz, y el código de red no se parecía en nada al resto del código del sistema.

En 1983, el grupo de Bill Joy en Berkeley publicó 4.2BSD con una idea nueva: tratar una conexión de red como un **descriptor de archivo**. Un socket se lee con `read()` y se escribe con `write()`, igual que un archivo o un pipe. Todo lo que ya sabías de Unix seguía valiendo.

Esa decisión —hacer que la red se pareciera a un archivo— fue tan buena que ganó por abandono. Se estandarizó en POSIX, la copiaron Windows (Winsock), Java, Python, Go y prácticamente todo lo demás. Cuarenta años después, la API que vas a usar hoy es reconociblemente la misma que escribieron en Berkeley.

Por eso hay rarezas: nombres cortos y crípticos (`bind`, `accept`) heredados de una época de compiladores con límites de identificadores, estructuras de direcciones incómodas, y funciones que hacen cosas sutilmente distintas según el tipo de socket. Estás usando una interfaz de los 80, y se nota.

Lo notable es lo bien que envejeció. Casi todo lo que vamos a ver funciona igual en C, y `AF_UNIX` —la variante local que vimos de pasada en la clase 12— usa exactamente las mismas llamadas.

> **Nota:** todos los ejemplos de esta clase son ejecutables. Los archivos `echo_server.py`, `echo_client.py` y `framing.py` acompañan la clase. Vas a necesitar dos terminales abiertas casi todo el tiempo.

Anatomía de un socket

Crear un socket es declarar qué tipo de comunicación querés:

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

Los dos parámetros:

Parámetro	Valor	Significa
Familia	`AF_INET`	IPv4
	`AF_INET6`	IPv6
	`AF_UNIX`	Local, por ruta de filesystem
Tipo	`SOCK_STREAM`	Flujo confiable (TCP)
	`SOCK_DGRAM`	Datagramas (UDP)

`AF_INET` + `SOCK_STREAM` es TCP sobre IPv4, la combinación de esta clase. UDP lo vemos en la clase 15.

El objeto que devuelve envuelve un descriptor de archivo. Podés verlo:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print(s.fileno())        # un entero, como el de open()
```

Ese número es de la misma naturaleza que el `3` que devuelve un `open()`. Es la herencia de Berkeley hecha visible.

Siempre cerrar

Un socket es un recurso del sistema operativo y hay que liberarlo. Como con archivos, `with` lo hace por vos:

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
    ...
# se cierra acá, incluso si hubo excepción
```

Usá `with` siempre. Un socket que queda abierto consume un descriptor, y los descriptores por proceso son un recurso limitado.

El cliente: tres pasos

El lado cliente es el más simple. Conectarse, hablar, cerrar.

```
#!/usr/bin/env python3
"""Cliente TCP mínimo."""
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
    s.connect(('localhost', 8080))      # handshake de tr
    s.sendall(b'hola mundo\n')
    respuesta = s.recv(4096)
    print(f'Recibido: {respuesta!r}')
```

Probalo: levantá `nc -l 8080` en otra terminal, corré el script, escribí algo en la terminal del `nc` y presioná Enter.

Qué hace cada línea:

- **`connect()`** dispara el handshake de tres vías de la clase

anterior. Bloquea hasta que la conexión esté establecida o falle.
- **`sendall()`** manda todos los bytes.
- **`recv(4096)`** bloquea hasta que lleguen datos, y devuelve **como máximo** 4096 bytes. Puede devolver menos.

Hay un atajo para el caso común:

```
with socket.create_connection(('localhost', 8080), timeou
    ...
```

`create_connection()` resuelve el nombre, prueba IPv4 e IPv6, y se conecta a la primera que ande. Casi siempre es lo que querés como cliente.

send() contra sendall()

Esta distinción causa bugs silenciosos.

`send()` devuelve **cuántos bytes mandó realmente**, que puede ser menos de los que le pediste si el buffer del kernel está lleno:

```
n = s.send(b'mensaje muy largo...')
print(n)     # puede ser menor que len(datos)
```

`sendall()` insiste hasta mandar todo, o lanza excepción. En la práctica:

```
s.sendall(datos)                # correcto
s.send(datos)                   # bug esperando a pasar
```

Usá `sendall()` salvo que tengas una razón concreta para manejar los envíos parciales vos.

El servidor: cinco pasos

El lado servidor tiene más ceremonia, y cada paso corresponde a algo real.

```
#!/usr/bin/env python3
"""Servidor TCP mínimo."""
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as
    servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUS
    servidor.bind(('0.0.0.0', 8080))
    servidor.listen(5)
    print('Escuchando en 0.0.0.0:8080...')

    while True:
        conn, direccion = servidor.accept()
        with conn:
            print(f'Conexión desde {direccion}')
            datos = conn.recv(4096)
            if datos:
                conn.sendall(datos)      # eco
```

bind: reclamar una dirección

```
servidor.bind(('0.0.0.0', 8080))
```

Le pide al sistema operativo el puerto 8080. Acá vuelve la distinción de la clase 12:

- `'127.0.0.1'` — solo alcanzable desde esta máquina
- `'0.0.0.0'` — todas las interfaces
- `''` (string vacío) — equivalente a `0.0.0.0`

Si el puerto está ocupado, `bind()` lanza `OSError: [Errno 98] Address already in use`.

SO_REUSEADDR: el que evita el error molesto

```
servidor.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADD
```

Sin esta línea, al matar el servidor y relanzarlo enseguida obtenés `Address already in use` durante uno a cuatro minutos, aunque no haya ningún proceso usando el puerto.

La causa es `TIME_WAIT`, el estado en que queda una conexión cerrada para absorber paquetes rezagados. `SO_REUSEADDR` le dice al kernel que igual te deje hacer bind.

Va **antes** del `bind()`; después no tiene efecto.

listen: abrir la cola

```
servidor.listen(5)
```

Convierte el socket en pasivo: pasa a aceptar conexiones en vez de iniciarlas. El número es el tamaño de la cola de conexiones pendientes de `accept()`.

Si la cola se llena, las conexiones nuevas se rechazan o se descartan. En un servidor real se usa algo mayor (`listen(128)`); para aprender, cualquier número sirve.

accept: obtener un socket nuevo

```
conn, direccion = servidor.accept()
```

Este es el punto que más confunde al principio.

`accept()` bloquea hasta que llegue un cliente, y devuelve **un socket nuevo y distinto** más la dirección del cliente. El socket original sigue escuchando; el nuevo es el que se usa para hablar con ese cliente.

```
servidor  (escucha en :8080, nunca transmite datos)
    │
    +-- accept() --> conn  (habla con el cliente A)
```

```
     +-- accept() --> conn  (habla con el cliente B)
```

Esto es coherente con la cuádrupla de la clase 12: el socket que escucha tiene solo (IP local, puerto local) definidos; cada `conn` tiene los cuatro valores completos y por eso identifica una conversación puntual.

Confundir los dos —mandar datos por el socket que escucha— es un error clásico y da errores desconcertantes.

---

recv: el que hay que entender bien

`recv(n)` es donde se rompen las intuiciones. Tres reglas:

1. Devuelve como máximo `n` bytes, y puede devolver menos.

No es un error: los datos pueden no haber llegado todos. El argumento es un tope, no una cantidad pedida.

2. Bloquea si no hay nada, pero devuelve apenas hay algo.

No espera a llenar el buffer. Si llega 1 byte, devuelve 1 byte.

3. Devuelve `b''` cuando el otro lado cerró.

Esto es la señal de fin de conexión, y hay que chequearla:

```
datos = conn.recv(4096)
if not datos:
    print('El cliente cerró la conexión')
    break
```

Sin ese chequeo, un bucle `while True` que lea de una conexión cerrada gira infinitamente consumiendo CPU. Es probablemente el bug número uno de quien empieza con sockets.

Verlo en vivo

Este ejemplo hace visible la lectura parcial pidiendo de a 4 bytes:

```
import socket

with socket.create_connection(('localhost', 8080)) as s:
    s.sendall(b'hola mundo\n')
    while True:
        pedazo = s.recv(4)          # de a 4 bytes a prop
        if not pedazo:
            break
        print(f'recv devolvió: {pedazo!r}')
```

Con `nc -l 8080` del otro lado escribiendo `respuesta larga`, vas a ver la respuesta llegar en pedazos de 4. Ningún mensaje se perdió: TCP entregó bytes, y vos elegiste de a cuánto leerlos.

El problema del framing

Acá está el tema central de la clase, y la deuda que dejamos pendiente en la clase 12.

TCP entrega un flujo de bytes ordenado y confiable. **No** preserva los límites de tus envíos. Tres `sendall()` pueden llegar como un `recv()`, o como cinco.

Comprobalo con el servidor eco corriendo:

```
s.sendall(b'HOLA')
s.sendall(b'COMO')
s.sendall(b'ESTAS')
```

El servidor probablemente lee `b'HOLACOMOESTAS'` de una vez.

Esto **no es un bug de TCP**: es exactamente lo que TCP promete. Si tu aplicación necesita mensajes, tenés que construirlos vos sobre el flujo. Eso se llama **framing**.

Estrategia 1: delimitador

Marcar el fin de cada mensaje con un byte que no aparezca en el contenido, típicamente `\n`:

```
HOLA\nCOMO\nESTAS\n
```

Es lo que hacen HTTP, SMTP, IRC y casi todos los protocolos de texto. Simple y depurable con `nc`.

El problema es qué pasa si el mensaje contiene el delimitador. Hay que escaparlo, o prohibirlo, o cambiar de estrategia.

La implementación necesita un **buffer**, porque un `recv()` puede traer media línea o tres líneas y media:

```
def recibir_lineas(sock):
    """Generador que produce líneas completas desde un so
    buffer = b''
    while True:
        pedazo = sock.recv(4096)
        if not pedazo:
            # Conexión cerrada: si quedó algo sin termina
            if buffer:
                print(f'Advertencia: datos incompletos {b
            return
        buffer += pedazo
        # Puede haber varias líneas completas en el buffe
        while b'\n' in buffer:
            linea, buffer = buffer.split(b'\n', 1)
            yield linea
```

Fijate que el `while` interno es necesario: si llegan tres líneas en un solo `recv`, hay que entregarlas las tres.

Python ofrece un atajo con `makefile()`, que envuelve el socket en un objeto tipo archivo:

```
with conn.makefile('rwb') as f:
    for linea in f:                        # itera por líne
        f.write(b'ECHO: ' + linea)
        f.flush()                          # importante: si
```

Es cómodo, pero conviene haber escrito el buffer a mano al menos una vez para saber qué esconde.

Estrategia 2: prefijo de longitud

Antes de cada mensaje, mandar su tamaño en un formato fijo:

```
[4 bytes: longitud][N bytes: contenido]
```

No tiene el problema del delimitador —el contenido puede ser cualquier cosa, incluso binario— pero exige leer una cantidad exacta de bytes, y `recv()` no garantiza eso. Hay que insistir:

```
import struct

def recibir_exacto(sock, n):
    """Lee exactamente n bytes, o devuelve None si la con
    datos = b''
    while len(datos) < n:
        pedazo = sock.recv(n - len(datos))
        if not pedazo:
            return None                    # cerró antes de
        datos += pedazo
    return datos

def enviar_mensaje(sock, payload: bytes):
    """Envía longitud (4 bytes, big-endian) seguida del c
    sock.sendall(struct.pack('!I', len(payload)) + payloa

def recibir_mensaje(sock):
    """Recibe un mensaje con prefijo de longitud."""
    cabecera = recibir_exacto(sock, 4)
    if cabecera is None:
```

```
        return None
    (longitud,) = struct.unpack('!I', cabecera)
    return recibir_exacto(sock, longitud)
```

Dos detalles que importan:

- **`recibir_exacto()` es imprescindible.** Sin ese bucle, un `recv(4)` puede devolver 2 bytes y el protocolo se desincroniza para siempre.
- **`'!I'` significa big-endian, entero sin signo de 4 bytes.** El `!` es el orden de bytes de red, que veremos enseguida.

Cuál usar

	Delimitador	Prefijo de longitud
Contenido binario	Necesita escape	Directo
Depurable con `nc`	Sí	No
Saber el tamaño de antemano	No hace falta	Obligatorio
Usado por	HTTP, SMTP, Redis	gRPC, WebSocket, protocolos binarios

Para protocolos de texto, delimitador. Para binario o mensajes grandes, prefijo de longitud.

Bytes contra strings

Los sockets mandan y reciben **bytes**, nunca strings. Python 3 es estricto:

```
s.sendall('hola')                  # TypeError
s.sendall('hola'.encode('utf-8'))  # bien
s.sendall(b'hola')                 # bien
```

Y al recibir hay que decodificar:

```
datos = s.recv(4096)
texto = datos.decode('utf-8')
```

Cuidado con un problema sutil: **un carácter UTF-8 puede ocupar varios bytes**, y `recv()` puede cortarlos por la mitad.

```
>>> 'ñ'.encode('utf-8')
b'\xc3\xb1'                          # dos bytes
```

Si un `recv()` termina justo entre esos dos bytes, `decode()` explota:

```
>>> b'\xc3'.decode('utf-8')
UnicodeDecodeError: unexpected end of data
```

Por eso conviene **decodificar después de armar el mensaje completo**, no sobre cada pedazo. Es otra razón para hacer bien el framing.

Como paliativo en logs y debugging:

```
datos.decode('utf-8', errors='replace')      # pone un 
```

Pero no lo uses para datos reales: esconde el bug en lugar de arreglarlo.

Orden de bytes en la red

Las arquitecturas no se ponen de acuerdo sobre cómo guardar un entero en memoria. Intel y ARM usan **little-endian** (byte menos significativo primero); otras usan **big-endian**.

Si mandás un entero crudo entre máquinas distintas, el número puede llegar al revés.

La convención de Internet es big-endian, llamado **orden de bytes de red**. Por eso `struct.pack('!I', n)` lleva `!`:

```
>>> import struct
>>> struct.pack('!I', 1)          # orden de red
b'\x00\x00\x00\x01'
>>> struct.pack('<I', 1)          # little-endian
b'\x01\x00\x00\x00'
```

Usá siempre `!` en los formatos de `struct` para datos que salen a la red. Con protocolos de texto no te afecta, pero apenas mandes un entero binario, sí.

Cerrar bien

`close()` cierra las dos direcciones. A veces querés cerrar solo una:

```
s.shutdown(socket.SHUT_WR)      # "no mando más, pero sig
```

Esto manda un FIN: el otro lado ve un `recv()` que devuelve `b''` y sabe que terminaste. Vos podés seguir recibiendo su respuesta.

Es el patrón para "mandá una petición, esperá la respuesta completa":

```
with socket.create_connection(('localhost', 8080)) as s:
    s.sendall(b'peticion completa')
    s.shutdown(socket.SHUT_WR)          # avisa que termi
    respuesta = b''
    while True:
        pedazo = s.recv(4096)
        if not pedazo:
            break
        respuesta += pedazo
```

Sin el `shutdown()`, si el servidor lee hasta que el cliente cierre, ambos

se quedan esperando: un deadlock distribuido.

Errores que hay que manejar

Un servidor que se cae porque un cliente se desconectó mal no sirve. Los habituales:

Excepción	Cuándo	Qué hacer
`ConnectionRefusedError`	No hay nadie escuchando	Reintentar con backoff
`ConnectionResetError`	El otro lado cortó abruptamente	Cerrar y seguir
`BrokenPipeError`	Escribir en conexión cerrada	Cerrar y seguir
`TimeoutError`	Venció `settimeout()`	Reintentar o abandonar
`OSError` Errno 98	Puerto ocupado	`SO_REUSEADDR`, u otro puerto

Un cliente con reintentos:

```
import socket
import time

def conectar_con_reintentos(host, puerto, intentos=5):
    for intento in range(1, intentos + 1):
        try:
            return socket.create_connection((host, puerto
        except (ConnectionRefusedError, TimeoutError) as
            espera = 0.5 * intento          # backoff lin
            print(f'Intento {intento} falló ({e}). Reinte
            time.sleep(espera)
    raise ConnectionError(f'No se pudo conectar a {host}:
```

Y el bucle del servidor, blindado:

```
while True:
    conn, direccion = servidor.accept()
    try:
        with conn:
            atender(conn)
    except (ConnectionResetError, BrokenPipeError) as e:
        print(f'Cliente {direccion} se desconectó: {e}')
    # el servidor sigue vivo para el próximo
```

Timeouts

Por defecto, un socket bloquea para siempre. Un `recv()` sin timeout contra un servidor que nunca responde cuelga el programa:

```
s.settimeout(5.0)             # 5 segundos
try:
    datos = s.recv(4096)
except TimeoutError:
    print('El servidor no respondió a tiempo')
```

Poner timeouts es lo que separa un cliente de juguete de uno usable. Es, además, la falacia número 2 de las que vimos en las manijas de la clase 12.

El límite: un cliente a la vez

Nuestro servidor tiene un problema grave. Mirá el bucle:

```
while True:
    conn, direccion = servidor.accept()
    with conn:
        atender(conn)             # <-- acá se queda
```

Mientras `atender()` conversa con un cliente, el bucle no vuelve a `accept()`. Los demás esperan en la cola de `listen()`, y cuando se llena, se los rechaza.

Compruébalo: levantá el servidor eco y conectá dos clientes. El segundo no recibe nada hasta que el primero se va.

Hay un detalle que confunde al diagnosticar esto. El segundo cliente **no** recibe un error: su `connect()` tiene éxito y `ss` muestra la conexión como `ESTAB`. Es que el handshake lo completa el kernel, no tu programa; la conexión queda esperando en la cola de `listen()` hasta que alguien llame a `accept()`.

O sea que "el cliente conectó bien" no significa "el servidor lo está atendiendo". Pueden pasar minutos entre una cosa y la otra.

Con un eco rápido casi no se nota. Con un cliente que se queda pensando diez segundos, el servidor está inutilizado diez segundos.

Es un servidor **secuencial**, y no sirve para nada real.

La solución vuelve a todo el bloque de concurrencia:

- Un **thread** por cliente (clase 10)
- Un **proceso** por cliente, con `fork()` (clase 4)
- Un **pool** de workers (clase 9)
- **Multiplexar** con `select()` y atender muchos en un solo hilo (clase 17)
- **Asyncio** (clase 19 en adelante)

Todo el bloque de concurrencia local existía, en parte, para poder resolver esto. La clase 14 lo retoma.

Conceptos clave

1. **Un socket es un descriptor de archivo**: la herencia de Berkeley que hace que la red se parezca al resto de Unix.
2. **`accept()` devuelve un socket nuevo**: el que escucha nunca

transmite datos.
3. **`recv(n)` devuelve *hasta* n bytes**: menos es normal, no es error.
4. **`recv()` devolviendo `b''` significa que el otro cerró**: chequealo siempre.
5. **Usá `sendall()`, no `send()`**: `send()` puede mandar de menos.
6. **TCP no tiene mensajes, tiene bytes**: el framing lo ponés vos, con delimitador o con longitud.
7. **`SO_REUSEADDR` antes del `bind()`**: evita el `Address already in use` de TIME_WAIT.
8. **Decodificá mensajes completos, no pedazos**: un carácter UTF-8 puede quedar partido entre dos `recv()`.
9. **Poné timeouts**: sin ellos, el programa puede colgarse para siempre.
10. **Un servidor secuencial atiende a uno solo**: la concurrencia es lo que viene.

Preparación para la próxima clase

En la **clase 14 (Servidores concurrentes)** vamos a resolver la limitación del final: un servidor que atienda a muchos clientes a la vez, con threads, con procesos y con un pool. Vamos a medir las tres estrategias y ver dónde se rompe cada una.

Para llegar preparado:

- Tené el servidor eco andando y entendido, línea por línea.
- Hacé el ejercicio de framing: es el que más se usa después.
- Convencete empíricamente de que el servidor secuencial bloquea al segundo cliente.

Referencias

- Beej's Guide to Network Programming - el mejor tutorial de sockets que existe; está en C, pero la API es la misma

- Socket Programming HOWTO - la guía oficial de Python, corta y muy buena
- Documentación del módulo `socket`
- Stevens, *UNIX Network Programming, Vol. 1* - la referencia canónica
- RFC 9293 - especificación actual de TCP
